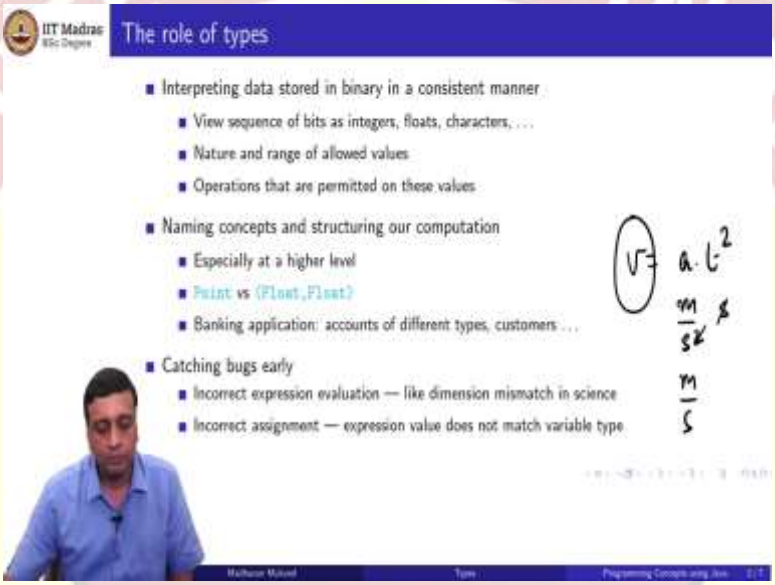
IIT Madras

ONLINE DEGREE

Programming Concepts Using Java
Online Degree Programme
B. Sc in Programming and Data Science
Diploma Level
Prof. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute

Lecture 2
Types

(Refer Slide Time: 00:22)



Let us take a closer look at types in programming languages. So, why do we need types? So, the first thing we said was that the storage in a computer is all binary we only have binary digits zeros and ones. So, everything is a bit sequence. So, there is no way of really recording meaningful value there except by interpreting it through a type. So, we have to decide whether this part of the memory should represent an integer or float what type of operations are allowed or in it and so on.

So, just to convert this kind of uniform storage of bits into some meaningful concepts that we can use externally we need to impose some notion of type. But the other part which is also useful is that by associating meaningful types with values we can more carefully record what are intent is when we are programming. So, we are programming at level where we are dealing with some abstract concepts and we want to reflect those concepts in the types that we are using.

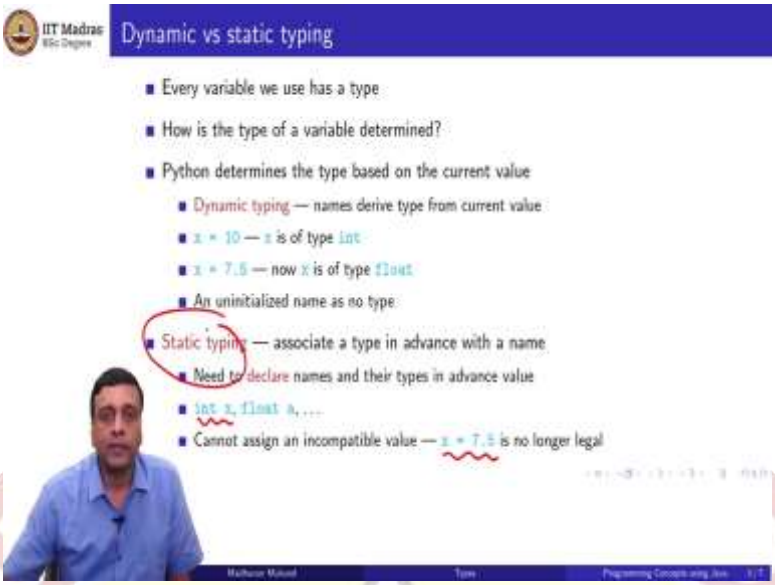
So, for instance if we are using a kind of geometric kind of application where we are dealing with points and so on. Even though a point is just at a very concrete level a pair of floats we might want to create an abstract notion of point as a type so, that we can keep track of those variables represents points and be able to read them off more clearly in a code. And of course we have a more complicated application something like a banking application we might have higher and order types.

I mean types which are more complex in nature. So, we might want a type to record a bank account and we might want to record customer as a different type and so on. So, we will come back to this next in this class. And the third thing that we mentioned all earlier also and we want to emphasize is the role of types in finding error. So, if your code is statically typed then even when you are reading it you can look for errors in typing.

And if it is dynamically typed then you do it when you are running it but in any case type errors are a bit like dimension errors when you are doing say a calculation in physics. So, supposing you want to know whether sorry. So, supposing you want to know whether velocity is actually acceleration times time then you can say that the dimension of acceleration is meters per second squared and time is second. So, then if I cancel I will get meters per second which is a correct dimension of the velocity.

Whereas it cannot be acceleration times time squared because then the two seconds will cancel out and you will have only meters. So, this is a kind of sanity check that you can do in a physics equation to check that the quantities that you are multiplying actually go together to produce the quantity that you want and in a similar way type checking does that.

(Refer Slide Time: 03:08)



So, let us look at the first part of it which is associating the meaningful value with a bit sequence. So, when I have a variable x in my program I want to know is it an int or a float or a char or whatever it is. So, how does the program determine this how does the programming language indicates what kind of type we are working with. So, in python this is done through the value which is associated with that variable.

So, python uses what is called dynamic typing. So, a name x does not in itself convey any type in python until it has a value. So, when we assign for instance x equal to 10 then at this point x becomes a value of type int. So, x on its own does not have any type it is the value which is stored in x which determines this type. So, python allows us to take the same x and later on change its value to 7.5 and in this process the type of x change.

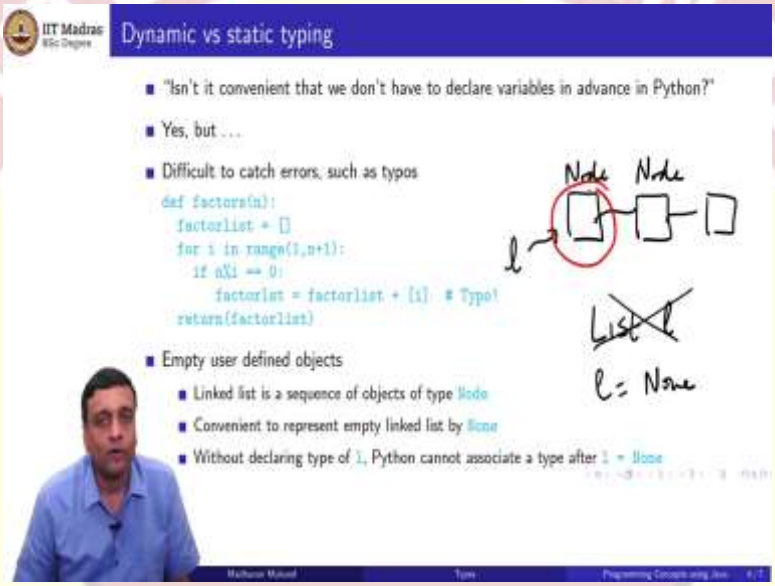
So, notice that x does not have a fixed type x's type depends on the value that it has and in particular when we start the first time we see an x in our program if it does not have any value associated with it has no type. So, an uninitialized name in python has no type and this has a consequence as we will. So, what is the option well the other option is to announce to the programming language what type each variable is?

So, this is what is done in languages like java which we will see and C and C++ another thing. So, you take a name and you associate a type with it upfront. So, you declare as it is called. So, the variable declaration, so, you have things like int x, this says x is a variable I am going to use and I am going to use it always as t ype integer. So, notice the

always. So, now I am unlike in the earlier case you cannot take this int x and convert it to 7.5 because that is no longer valid.

So, similarly you can say for any type you can say as a type float and so on. So, the main point is that this is now static. Static in the sense that the type of value of a variable is fixed once and for all through its declaration and you cannot change it just by assigning a different value to it. In fact you are not allowed to ideally assign noncompatible values given the type of the variable.

(Refer Slide Time: 05:28)



So, when people argue about which programming language is easiest to learn. So, one of the arguments made in favor of python is that it is very convenient because you do not have to do all this bureaucracy of declaring variables in advance. Now this is indeed true it does cut down the syntax of the language and make it easier to approach for a new program because you do not have all these mysterious int x and float a before you start doing anything meaningful by way of computation.

But it is not necessarily a great thing to have this feature in your programming language. So, let us look at an example. So, the first is what happens when you make a mistake by just accidentally mistyping a varaible. So, here is a trivial function which computes the list factors of a given number n. So, it starts with an empty list factor list and it runs through all the numbers from 1 to n.

So, remember that range goes one less. So, n + 1 for n. So, it goes from one to n and wherever you find a factor something which divides in without leaving the remainder added to the factors. All well and good except there is a problem and that is that in this update to factor list we have unintentionally written factor list without an i on the left. So, there is a type of this maybe something which you catch or do not catch when you are reading it.

But the point is python will not complain about this at all because this is a perfectly valid python program. It does not do what you intend and it might take you a long time to figure out why it is doing something wrong. But essentially what is happening is that each time you come here factor list remains empty because you have never really updated it. You take the latest factor and you assign factor lst the spurious variable to be the latest factor of found as a singleton list.

So, at the end of this, this will basically have the largest factor namely n. So, this particular thing will always end up with this is the return value. Now this problem is there because python has no way of knowing when you use a variable for the first time whether it is a new variable or an old variable or not because you have not declared and its type remember is inherited in some sense from the value it is associated.

So, that is really the problem that you know if you do not announce in advance you cannot catch errors like this. So, that is an error. The other problem with this dynamic typing is the fact that you do not have a type if you do not have a value. Now most situations you would not want this. I mean there is no reason why you want to use I am initialized integer or an uninitialized float anywhere.

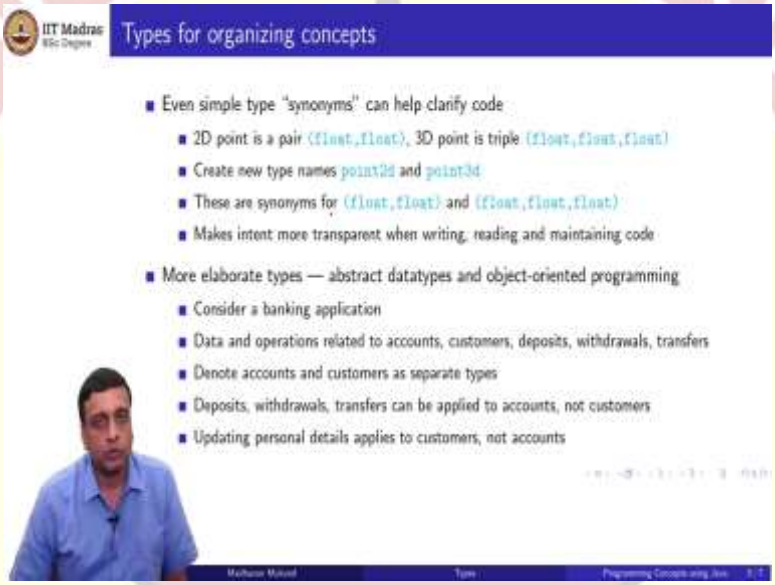
But if you have more complex things like these abstract data types like linked list. So, remember that in a linked list we typically have a sequence of these nodes and then we have our variable of say l pointing to this sequence of nodes. So, these are all nodes. Now if I could tell you in advance that l is of type list. So, this is a declaration which you do not have in python.

Then I could meaningfully assign l to be the value none to denote the empty list. But this is not possible in python because I do not have this. So, now in python if I say l is equal

to none then l has no type because none has no type. So, I am forced to actually use some kind of a dummy node and make l point to it and have some convention by which I say a node and would say the value is none denotes an empty list.

So, I have to really create if you want to think about that create a non-empty list with some special property to denote an empty list and this becomes very clunky to program with if you are used to programming in a statically typed language where you can define the type and then associate a non or a nil value with that value with that type with that variable.

(Refer Slide Time: 09:28)



So, the second utility of using types we said was to align the program more closely to the concepts that we are trying to express. So, this is an example we looked at the beginning of today's lecture. So, we said if you are dealing with geometry objects like types you might have two dimensional points, three dimensional points. So, two dimensional point is a pair of floats a three dimensional point is just a triple of floats.

But if you think about them as new types point 2d and point 3d it becomes clearer to you as your reading it which variables are doing and for the reader of the program or the maintenance of the program also it is easier. So, here we are really using some kind of a synonym we are not creating a new type as okay it is not like a stack with pop or push is just saying that we are renaming a particular thing a pair of floats to be point 2d.

So, when I read point 2d's more clear what the intent is than to read pair of flow. So, this is from a readability perspective. But as we know we have also more complicated scenarios which we want where we want to create these more complicated types with designated operations. So, we have typically looked at things like stacks and queues and priority queues and so on.

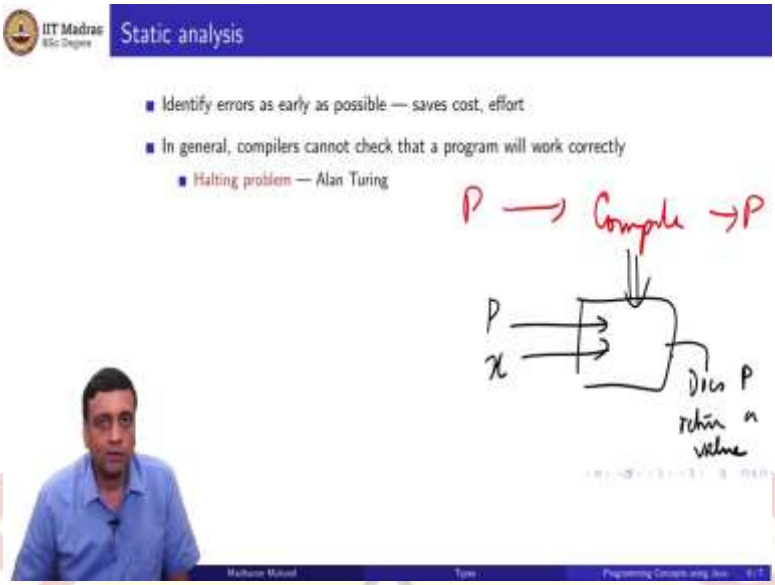
But there is nothing to prevent this from being at an arbitrary level of sophistication. So, imagine that you are writing a large banking application. So, within the banking world there are some well defined entities which you have to typically manipulate. So, there will be bank accounts of course there will be customers and then the types of operations that you do will also be classified into different things.

They will be deposits they will be withdrawals every transfers and each of them carries some data. When I do a transfer a transfer will consist of several things it will consist of the time of the transfer the account from the transfer to which it is transferred the amount of the transfer and so on even maybe the currency of the transfer. So, these are all things which have their own associated with it data and possibly operations associated with it.

So, if you could create in this abstract data type world higher level types denoting these different objects differently. So, you can have accounts and you can have customers do not behave the same. So, if you want to do withdrawals and deposits and so on you cannot apply to a customer but of course you could apply to an account. Similarly if you want to update say the KYC details of a customer you could apply to a customer object but not to an account object.

So, in this way by having types which reflect the ground truth of what you are trying to manipulate in a more direct you can make your program easier to maintain and also guarantee correctness to make sure that you are not doing the wrong thing with the wrong object.

(Refer Slide Time: 12:09)



So, this brings us to the idea of doing wrong things. So, one thing that static typing allows us to do is to do what is called static analysis by just reading the code the compiler is able to actually detect errors early. Now and anything that we do the earlier we detect an area the earth better it is I mean whether it is constructing a building or whether it is writing a program.

If you find out the fault after the program is built or the building is built then it is much more difficult to break it down and then to redo the thing than to find out right at the beginning there is something flawed of the design. So, in every case early detection of errors saves cost in real cost and also effort and time. Now the reason this is important is because there is a fundamental limitation of programs which is that programs cannot check themselves.

So, this is the content of Alan Turing's famous result concerning the impossibility of the halting problem. So, remember that we have compilers. So, we have compilers it would take a program and produce a program. So, we do have programs which read other programs and they can do something meaningful a compiler can check that the input program is syntactically correct and if it is correct it can translate it into another language machine language.

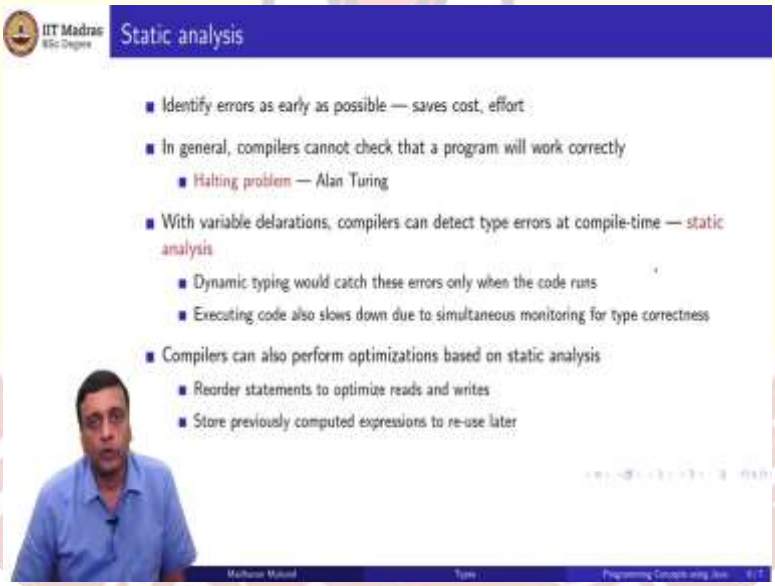
So, now what I want to do is write a program which will take as input another program and some value that is input to that program. So, I want to simulate be on x and I want thus p return a value. So, this is not even asking whether p does a correct thing on x it is

just asking whether p produces some output on x does it halt that hence the halting problem or does it just go into an infinite loop and not come out at all.

This looks like something that should be doable if we can compile a program and decide whether it is correct and then finally we can even interpret it and run it. Why cannot we examine it and figure out whether or not it will at least terminate on a given input. But what Alan Turing proved was this is not possible. It is not possible to write such a kind of higher level object than a compiler which can actually analyze arbitrary programs and decide whether their behaviour is valid or not valid.

So, given that we cannot do this in general. So, we cannot hope to check programs even for something as basic as halting forget about correctness we need to provide as many mechanisms as possible to make sure that we do not make mistakes and one of the easier things to fix is this type error. So, that is why we use typing.

(Refer Slide Time: 14:54)



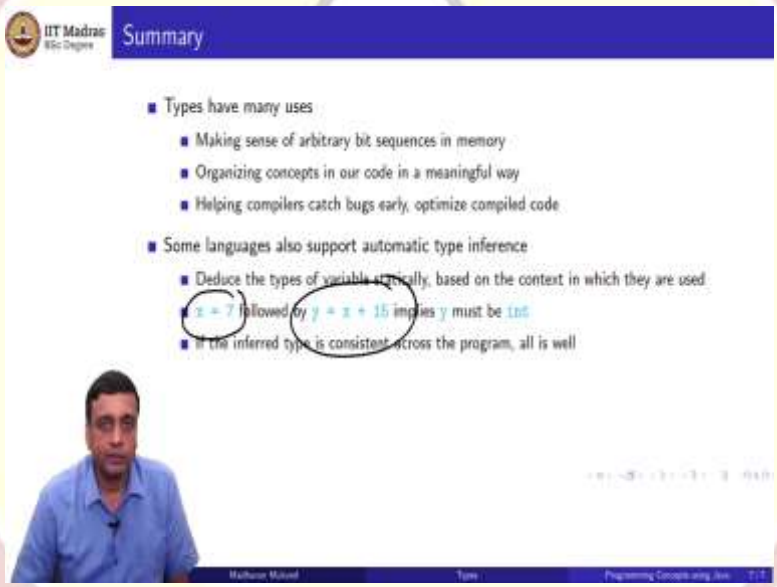
So, with variable declarations and if we have that typing a compiler can easily detect whether there are such errors. Now we can also do this with dynamic typing. So, python obviously finds errors as it runs but then you can see that python is actually monitoring these errors as they come. So, some of the errors that python finds are runtime errors you are trying to divide by zero or something.

But if it is a type like you are trying to combine a list and an integer into a new list. Then this kind of error can be caught much easier statically at compiled and even before your

program runs a compiler can say this is not going to work. So, when you do it dynamically you incur an overhead because you have to monitor these errors as you are running whereas many of these errors can actually be caught in advance if you do static typing.

The other thing which we will not address much is that you can also do some optimization. So, a compiler can now take a look at some of the code and do some it can for example check that certain values are used somewhere else and computer an intermediate thing it can it can figure out some relationships between parts of objects and so on. So, we will not get into this but this is one of the features of having a good typing mechanism.

(Refer Slide Time: 16:07)



So, to summarize what we have seen is that there are at least three different ways in which typing helps us program. The first is absolutely essential which is that we need to make sense out of this bit world that we live in all right. So, the computer stores everything as sequences a bit and if we did not impose a certain type discipline on top of these storage objects we would not be able to say that this is a deterrent that is a float because internally everything looks the same to the computer.

So, if I take a sequence I could interpret it whichever way you tell me to and then it makes a different value. So, the way I represent zero may or may not be the same as an end as a flow. The other thing is that we can use these types to make our programs more transparent in the sense that we can create types which reflect the concepts we are

manipulating. So, we saw examples of points a geometric program or bank accounts and customers and so on.

And finally typing helps compilers detect errors at an early stage which is always a good thing. Now one thing which we should not assume is that so, we mentioned that in a statically typed language like C or java usually you declared the types and only then you get to know the types. Now it is actually not true that you need to declare types if you have a statically type language. So, you can also if you know that the types are static. So, in python the problem is the types are not static.

So, we saw that you can say x equal to 10 and then you can say x equals 7.5. So, there is no consistency of the type of x it depends on the value that is stored. But supposing I tell you that every variable has a fixed type and the types should be uniform across the thing. Then I can start inferring the type based on the context. For instance if I see something like x equal to 7 somewhere in my program this tells me that x in this context is certainly an integer because there assign an integer value.

So, let me assume that everywhere it is an integer. Now somewhere if I find out that it is not an integer then there is an error and I can flag it. Similarly if I see something like y equal to x plus 15 and I have already deduced that x is an integer I can also deduce y is an integer. So, I can make these deductions and this is called type inference and inferring from the context to and it is used.

And if this inference is consistent that is if I can consistently assign a type to every variable then I do not need the programmer to tell me the types. So, this is very interesting and this is there in many languages it is not there in java but this is also there. So, static typing is important but type declarations are not always required for static typing if you have a more sophisticated way of inferring types but in our context static typing goes hand in hand declaring variables.